KernelSim: OS Scheduling and Resource Management Simulator

1. Abstract

KernelSim is an educational OS lab with a terminal-based C++17 application and a zero-dependency browser companion. The browser companion follows a simple process-input → solve → Gantt chart workflow inspired by the referenced Process Scheduling Solver, and extends it with six scheduling policies, side-by-side comparison, extra metrics, CSV/JSON input/output, a concept guide, and Bankers resource-safety analysis. The C++ application demonstrates selected operating-system algorithms through object-oriented design. It simulates five CPU scheduling policies, generates Gantt timelines, calculates scheduling metrics, compares algorithm performance, and evaluates resource allocation using the Banker's safety algorithm. The project uses only the C++ standard library and is designed to be understandable and testable at second-year BTech CSE level.

2. Problem statement

CPU scheduling affects process waiting time, turnaround time, response time, and overall utilization. Different scheduling policies behave differently for the same workload. Resource allocation also requires care: granting every currently possible request can leave a system in an unsafe state. KernelSim provides repeatable simulations that make these effects visible and measurable.

3. Objectives

1. Implement FCFS, SJF, SRTF, Priority, and Round Robin.
2. Track process state and execution intervals.
3. Calculate per-process and aggregate metrics.
4. Compare algorithms against the same workload.
5. Implement the Banker's safety test and resource-request validation.
6. Demonstrate abstraction, inheritance, polymorphism, encapsulation, composition, STL, and file handling.
7. Deliver a portable terminal application with tests and reproducible sample inputs.

4. Scope and assumptions

The simulator uses one CPU and integer time units. Every process has one CPU burst; there is no I/O blocking. Context-switch overhead is zero. SJF and Priority Scheduling are non-preemptive, SRTF is preemptive, and Round Robin uses a configurable positive time quantum. A smaller numeric priority means a higher priority. The resource-allocation module is independent of the CPU timeline and demonstrates the Banker's algorithm rather than a real kernel resource manager.

5. Browser companion

The web/ directory contains a static browser application that works offline and requires no package installation. It provides an editable process table, six scheduling policies (including preemptive Priority), configurable Round Robin quantum, Gantt chart, per-process and aggregate metrics, algorithm comparison, workload import/export, dark mode, and a concept guide. Its separate Bankers lab displays Allocation, Maximum, Need and Available, computes a safe sequence, and only commits a request if the candidate allocation remains safe. Static deployment is supported by GitHub Pages, Netlify, Cloudflare Pages, or any static host.

6. Functional requirements

• Accept and validate process IDs, arrival times, burst times, and priorities.

- Execute five scheduling policies.
- Record process states and the Gantt timeline.
- Calculate completion time (CT), turnaround time (TAT), waiting time (WT), and response time (RT).
- Calculate average WT, average TAT, average RT, CPU utilization, and throughput.
- Compare policies on a shared workload.
- Create, save, and load process workloads from the terminal.
- Load a sample Banker's algorithm scenario.
- Calculate Available and Need, evaluate a safe sequence, and test resource requests.
- Reject a request that violates declared need, exceeds availability, or creates an unsafe state.
- Export the latest scheduling result.

7. Metric definitions

For each process:

- `TAT = CT - AT`
- `WT = TAT - BT`
- `RT = first_start_time - AT`

Aggregate metrics:

- Average WT = sum of process waiting times / number of processes.
- Average TAT = sum of process turnaround times / number of processes.
- Average RT = sum of process response times / number of processes.
- CPU utilization = total busy CPU time / elapsed time × 100%.
- Throughput = completed processes / elapsed time.

Elapsed time starts at time zero, so an idle interval before the first arrival contributes to elapsed time but not busy time.

8. Scheduling algorithms

First Come First Served

Selects the earliest-arrived eligible process and runs it until completion. It is simple, but a long process can delay shorter jobs behind it.

Shortest Job First

Selects the eligible process with the smallest original burst time. It is non-preemptive, so a newly arriving short process does not interrupt a process already executing.

Shortest Remaining Time First

Selects the eligible process with the smallest remaining burst time and can preempt the current process when a better candidate arrives.

Priority Scheduling

Selects the eligible process with the smallest numeric priority value. This implementation is non-preemptive. Starvation is possible for low-priority processes when higher-priority work continues to arrive.

Round Robin

Processes receive up to a configured time quantum in FIFO ready-queue order. In this simulator, arrivals at a time-slice boundary are admitted before the expired process is appended to the back of the queue.

9. Resource allocation

For each process and resource type:

```
Need = Maximum - Allocation

Available = Total - sum of current allocations
```

The safety algorithm copies Available into Work, then repeatedly selects an unfinished process whose Need is no greater than Work. It simulates that process finishing and returning its allocated resources to Work. If every process can finish, the state is safe and the selected order is a safe sequence.

For a request, KernelSim checks the process's remaining Need and current Available resources. It then evaluates a candidate allocation. The candidate replaces the live allocation only if it is safe; otherwise the live state remains unchanged. This implements transactional rejection of an unsafe request.

10. Object-oriented design

Class / type	Responsibility
`Process`	Encapsulates process attributes, remaining time, timestamps, and state.
`SchedulingPolicy`	Abstract strategy interface for selecting a ready process.
`FCFSPolicy`	First-arrived candidate.
`SJFPolicy`	Shortest original burst.
`SRTFPolicy`	Shortest remaining burst and preemption.
`PriorityPolicy`	Highest priority according to the documented convention.
`RoundRobinPolicy`	FIFO selection and time quantum.
`SchedulingEngine`	Advances time, handles arrivals, dispatch, preemption, completion, and metrics.
`SimulationResult`	Stores timeline, per-process metrics, and aggregate metrics.
`ResourceManager`	Maintains resource matrices and evaluates safety and requests.
`FileManager`	Loads scenario files and exports reports.
`ConsoleUI`	Handles user input and presentation.

OOP mapping

- Encapsulation: process attributes and resource matrices are private.
- Abstraction: `SchedulingPolicy` defines the common scheduling interface.
- Inheritance: concrete policies derive from `SchedulingPolicy`.
- Runtime polymorphism: the engine invokes virtual policy operations through a base-class reference.
- Composition: `SimulationResult` contains timeline segments and process result records.
- Constructors: validate process and resource configurations.
- Virtual destructor: permits safe destruction through a base-class pointer.
- STL: vectors and algorithms support the timeline, process list, ready queue, and matrix calculations.
- File handling: scenarios and reports are stored as plain text.

11. Testing and verification

The test executable checks thirteen scenarios:

1. FCFS completion times and aggregate metrics.
2. SJF non-preemptive selection.
3. SRTF preemption and completion times.
4. Priority ordering.
5. Round Robin time-quantum rotation.
6. Idle-time accounting and CPU utilization.
7. Safe sequence for the standard five-process Banker's example.
8. Rejection of an unsafe request without mutation.
9. Acceptance of a safe request.
10. Rejection of an invalid request.
11. Rejection of a zero-burst process.
12. Rejection of a request exceeding remaining need without mutation.
13. Metric and timeline invariants across all five scheduling policies.

Additional manual checks should include malformed input files, duplicate IDs, invalid burst times, a Round Robin quantum of one, and workloads with simultaneous arrivals.

12. Build and deployment

The C++ application uses C++17 and the standard library only. Build with `build.bat` on Windows with MinGW-w64, `build.sh` on a GCC-based shell, or CMake. The browser companion runs from `web/index.html` and needs no build step. Run `KernelSimTests` before demonstrating the application. Launch from the project root so the `data` and `results` directories are found predictably.

13. Limitations and future work

KernelSim does not implement a real kernel, CPU context-switch costs, I/O blocking, multicore scheduling, actual operating-system threads, memory paging, or persistent resource-state updates. Potential future extensions include aging to reduce priority starvation, an I/O-blocked process state, context-switch overhead, and a semaphore-based Producer–Consumer simulation.

14. Six-minute demonstration plan

1. Introduce the abstract policy interface and scheduling engine.
2. Run FCFS, SRTF, and Round Robin on the same sample workload.
3. Explain Gantt timelines and compare waiting/turnaround/response metrics.
4. Load the Banker's example and show its safe sequence.
5. Submit the request `P0: [0, 0, 2]` and explain why it is rejected as unsafe.
6. Show one automated test and explain how a new scheduling policy can be added without rewriting the engine.

15. Conclusion

KernelSim combines core CPU scheduling algorithms with resource-allocation safety analysis in a modular, testable terminal application. Its focus is algorithmic correctness and defensible OOP design rather than visual complexity. The implementation remains small enough for an individual second-year student to understand and explain.